Algoritmia y Estructuras de Datos Avanzadas.
Año Académico 2026-27

Práctica 2: Algoritmos codiciosos.

Fecha de entrega:

- **Grupo 126:** 9 de noviembre 2026 a las 23:59 horas.
- **Grupo 127:** 10 de noviembre 2026 antes del inicio de la clase.

¡Atención! Muchas de las funciones de esta práctica (y de las demás) se pasarán por un script de corrección automática. Esto quiere decir que tenéis que escribir escrupulosamente las indicaciones en lo que se refiere a nombres de funciones, parámetros, valores de retorno, ficheros, etc. Las funciones no deben escribir nada en pantalla (si ponéis algunos `print` para hacer depuración, no se os olvide eliminarlos antes de entregar). El fichero debe contener únicamente las importaciones necesarias y las funciones solicitadas. No debe contener código de prueba ni instrucciones que se ejecuten automáticamente al importar el módulo. No seguir estas indicaciones puede resultar en una penalización o, en el peor de los casos, en la no corrección de la práctica.

I. Problema del cambio de monedas (**3,5 puntos**)

Dado un conjunto de monedas y una cantidad objetivo se desea obtener exactamente dicha cantidad. Se supondrá que existe un número ilimitado de monedas de cada moneda, que las monedas son enteros positivos no repetidas y que el sistema no contiene necesariamente una moneda de valor 1.

I.A Solución codiciosa (**0,65 puntos**)

La estrategia codiciosa seleccionará, en cada paso, la moneda de mayor valor que no supere la cantidad restante.

I.A.1 Escribir la función:

```
greedy_change(coins: List[int], amount: int) -> Tuple[List[int], int]
```

La función devolverá una tupla `(selected, remainder)`, donde `selected` contiene las monedas elegidas y `remainder` la cantidad que no ha podido completarse. Si `remainder == 0`, la solución es exacta. La función no deberá asumir que la lista `coins` está ordenada.

Ejemplo 1: `greedy_change([50, 20, 10, 5, 2, 1], 87)` deberá devolver ([50, 20, 10, 5, 2], 0)

Ejemplo 2: `greedy_change([4, 3, 1], 6)` deberá devolver ([4, 1, 1], 0)

Nota: en el ejemplo 2 la solución es exacta, pero no óptima, ya que [3, 3] utiliza menos monedas.

Ejemplo 3: `greedy_change([5, 3], 7)` deberá devolver ([5], 2)

Nota: la solución codiciosa no alcanza la cantidad objetivo.

I.B Solución mediante programación dinámica (**0,90 puntos**)

Se implementará una solución óptima que permita determinar el menor número de monedas necesario para obtener la cantidad objetivo.

I.B.1 Escribir la función:

```
optimal_change(coins: List[int], amount: int) -> List[int] | None
```

La función devolverá una lista con una combinación que utilice el menor número posible de monedas o None si no existe una combinación exacta.

Si existen varias soluciones con el mismo número de monedas, las monedas de cada combinación se ordenarán de mayor a menor y se devolverá la combinación lexicográficamente mayor.

Para `amount == 0` deberá devolverse `[]`. No deberá utilizarse búsqueda exhaustiva.

Ejemplo 1: `optimal_change([4, 3, 1], 6)` puede devolver [3, 3]

Ejemplo 2: `optimal_change([5, 3], 7)` deberá devolver None

I.C Comparación de las soluciones (**0,45 puntos**)

I.C.1 Escribir la función:

```
compare_change(coins: List[int], max_amount: int) -> Dict[str,
List[int]]
```

La función evaluará todas las cantidades entre 1 y `max_amount`, ambos incluidos, y devolverá el siguiente diccionario:

```
{
    "optimal": [...],
    "suboptimal": [...],
    "greedy_failed": [...],
    "impossible": [...]
}
```

- **`optimal`**: cantidades para las que *greedy* es exacto y utiliza el mínimo número de monedas.
- **`suboptimal`**: cantidades para las que *greedy* es exacto, pero utiliza más monedas que la solución óptima.

- **`greedy_failed`**: cantidades para las que *greedy* no es exacto, aunque existe otra combinación exacta.
- **`impossible`**: cantidades para las que no existe ninguna combinación exacta.

I.D Análisis experimental (**0,75 puntos**)

Analizar los siguientes sistemas para cantidades comprendidas entre 1 y 500:

```
[200, 100, 50, 20, 10, 5, 2, 1]
[10, 5, 2, 1]
[25, 10, 4, 1]
[4, 3, 1]
```

En memoria.pdf deberán incluirse:

- El número y porcentaje de cantidades clasificadas como optimal, suboptimal, greedy_failed e impossible.
- Una tabla resumen con dichos resultados.
- Una comparación experimental de los tiempos de ejecución de `greedy_change` y `optimal_change`.
- Una breve discusión sobre el compromiso entre eficiencia y calidad de la solución.

Cuestiones (**0,75 puntos**):

1. *¿Cuál es el criterio codicioso utilizado?*
2. *¿Funciona correctamente para todos los sistemas monetarios?*
3. *¿Cuál es la complejidad temporal de la solución codiciosa?*
4. *¿Cuál es la complejidad temporal y espacial de la programación dinámica?*
5. *¿Qué ventajas presenta la solución codiciosa aunque no siempre garantice el óptimo?*

II. Selección de actividades (**2,5 puntos**)

Cada actividad se representará mediante una tupla `(identifier, start, end)`. Dos actividades serán compatibles cuando no se solapen. Una actividad que comienza exactamente cuando termina otra se considerará compatible.

II.A Implementación (**1 punto**)

La estrategia ordenará por tiempo de finalización ascendente. En caso de empate se utilizarán, sucesivamente, el tiempo de inicio y el identificador.

II.A.1 Escribir la función:

```
activity_selection(activities: List[Tuple[str, int, int]]) ->
List[Tuple[str, int, int]]
```

La función devolverá una nueva lista con el máximo número posible de actividades compatibles, ordenadas según su ejecución. No deberá modificar la lista recibida.

Ejemplo 1: Para [("A",1,4), ("B",3,5), ("C",0,6), ("D",5,7), ("E",8,9), ("F",5,9)] deberá devolver [("A",1,4), ("D",5,7), ("E",8,9)]

II.B Generación de actividades (**0,35 puntos**)

II.B.1 Escribir la función:

```
generate_activities(n: int, seed: int = 42) -> List[Tuple[str, int, int]]
```

- `start` pertenecerá a [0, 10000].
- `duration` pertenecerá a [1, 100].
- `end = start + duration`.
- Los identificadores tendrán el formato A0, A1, ..., A(n-1).
- La semilla permitirá reproducir las entradas.

II.C Análisis experimental (**0,55 puntos**)

Ejecutar `activity_selection` con 100, 500, 1000 y 5000 actividades. Cada medición temporal se repetirá al menos diez veces sobre las mismas entradas.

- Gráfica del tamaño de entrada frente al tiempo.
- Gráfica del tamaño de entrada frente al número de actividades seleccionadas.
- Gráfica del porcentaje de actividades seleccionadas.

Cuestiones (**0,6 puntos**):

1. *¿Cuál es el criterio codicioso utilizado?*
2. *¿Por qué seleccionar la actividad que termina antes deja más posibilidades?*
3. *¿Qué podría suceder si se eligiese primero la actividad que comienza antes?*
4. *¿Qué podría suceder si se eligiese primero la actividad de menor duración?*
5. *¿Cuál es la complejidad temporal y espacial?*

III. Planificación de tareas con fechas límite (**3 puntos**)

Cada tarea se representará mediante una tupla `(identifier, profit, deadline)`. Todas las tareas tienen duración unitaria, solo pueden ejecutarse una vez y proporcionan beneficio únicamente si finalizan dentro de su fecha límite.

III.A Implementación (**1,25 puntos**)

1. Ordenar las tareas por beneficio decreciente.
2. Asignar a cada tarea el último intervalo libre que no supere su deadline.
3. Descartar la tarea si no existe ningún intervalo válido.

En caso de empate se utilizarán, sucesivamente, el menor deadline y el identificador en orden lexicográfico.

III.A.1 Escribir la función:

```
job_scheduling(
    jobs: List[Tuple[str, int, int]]
) -> Tuple[List[Optional[Tuple[str, int, int]]], int]
```

Se devolverá `(schedule, total_profit)`. La posición `0` de `schedule` representa el intervalo 1. Los intervalos vacíos se representarán mediante `None`. La longitud de `schedule` será igual al máximo `deadline`. Para una lista vacía se devolverá `([], 0)`.

Ejemplo 1: Para [(A,100,2),(B,19,1), (C,27,2), (D,25,1), (E,15,3)], el resultado será ([(C,27,2), (A,100,2), (E,15,3)], 142)

III.B Generación de tareas (**0,35 puntos**)

III.B.1 Escribir la función:

```
generate_jobs(n: int, seed: int = 42) -> List[Tuple[str, int, int]]
```

- `profit` pertenecerá a [1, 1000].
- `deadline` pertenecerá a [1, max(1, n // 10)].
- Los identificadores tendrán el formato J0, J1, ..., J(n-1).

Número de tareas	Rango de deadlines
100	1 a 10
500	1 a 50
1000	1 a 100
5000	1 a 500

III.C Análisis experimental (**0,65 puntos**)

Ejecutar `job_scheduling` con 100, 500, 1000 y 5000 tareas.

- Gráfica del número de tareas frente al tiempo.
- Gráfica del número de tareas frente al beneficio total.
- Gráfica del número de tareas frente al porcentaje seleccionado.

Cuestiones (**0,75 puntos**):

1. *¿Cuál es el criterio codicioso utilizado?*
2. *¿Por qué se procesan primero las tareas con mayor beneficio?*
3. *¿Cómo afecta la distribución de deadlines?*
4. *¿Qué podría suceder si se ordenasen únicamente por deadline?*
5. *¿Cuál es la complejidad temporal y espacial?*

IV. Cuestiones generales (**0,5 puntos**)

1. *¿Qué rasgos permiten afirmar que los tres algoritmos estudiados pertenecen al paradigma codicioso?*
2. *¿Qué diferencias existen entre los tres algoritmos en complejidad temporal y espacial?*

V. ¿Qué tengo que entregar?

i) Un fichero **`p2.py`** que contenga:

- Las funciones solicitadas en el apartado I: `greedy_change`, `optimal_change` y `compare_change`.
- Las funciones del apartado II: `activity_selection`, `generate_activities`.
- Y las funciones del apartado III: `job_scheduling`, `generate_jobs`.

¡Atención! El fichero debe contener sólo las funciones, ningún script de prueba.

ii) Un fichero **`memoria.pdf`** con las respuestas a las cuestiones planteadas a lo largo de los apartados I, II, III y IV.

0,5 puntos para:

- Organización y claridad del código.
- Claridad y presentación de memoria.pdf.
- Coherencia entre código, resultados y respuestas.
